AL-2002 - Artificial Intelligence Lab
Lab # 6

Objectives:

- CSP (Arc Consistency + Backtracking Seach/Forward Checking)

Note: Carefully read the following instructions (*Each instruction contains a weightage*)

1. First think about statement problems and then write your program.
2. Write Program in multiple cells of the same notebook**.**
3. **Must Use Functions and Exception handling for each question, where they can be applied.**
4. Do not copy from any source otherwise you will be penalized with negative marks.
5. Add your source code in word document and attach output screenshots.
6. Upload word file and ipynb file.
7. Please submit your **Both files** with this naming convention ROLLNO_SECTION_LABNO. **(Do not upload zip file).**
8. Submit your lab on Google Classroom.

Question 1:

Implement constraint satisfaction algorithm, by coloring a map of Australia each state region either Red, Green, or Blue in such a way that no two neighboring regions have the same color. (a) Showing each of its states and territories. (b) The map-coloring problem represented as a constraint graph.

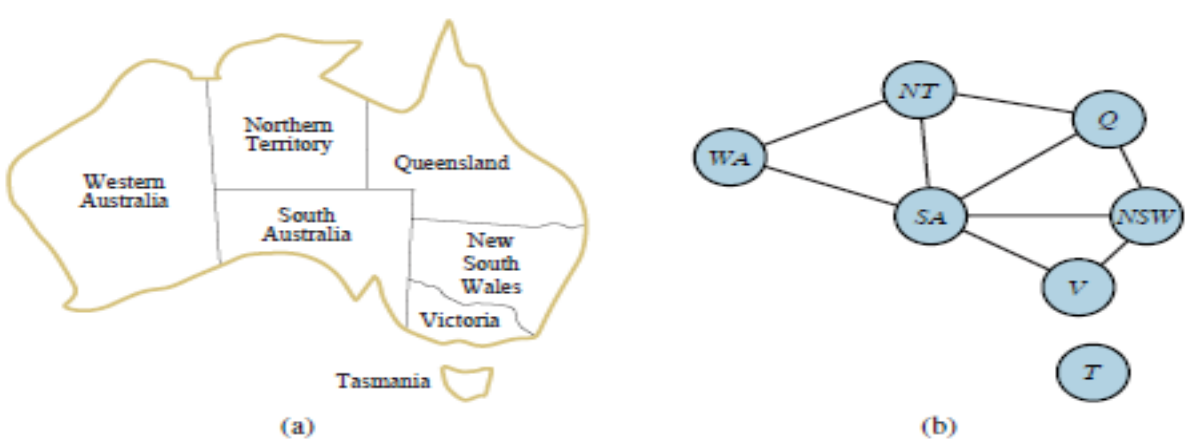

Figure 6.1 (a) The principal states and territories of Australia. Coloring this map can be viewed as a constraint satisfaction problem (CSP). The goal is to assign colors to each region so that no two neighboring regions have the same color. (b) The map-coloring problem represented as a constraint graph.

- A CSP is composed of:
    - A set of variables $X_1, X_2, \ldots, X_n$ with domains (possible values) $D_1, D_2, \ldots, D_n$
    - A set of constraints $C_1, C_2, \ldots, C_m$
    - Each constraint C_i limits the values that a subset of variables can take e.g., $V_1 \neq V_2$

In our example

- Variables: *WA, NT, Q, NSW, V, SA, T*
- Domains: $D_i = \{red, green, blue\}$
- Constraints: adjacent regions must have different colors
    - E.g., $WA \neq NT$ (if the language allows this) or
    - (*WA*, *NT*) in {(red, green), (red, blue), (green, red), (green, blue), (blue, red), (blue, green)}

Question 2:

Implement the **Backtracking Search** and **Forward Checking** of Australia Map given in Task 1. The backtracking algorithm as given below uses the simplest strategy for selecting the unassigned variables in a static ordering. Also, implement an intuitive idea, choosing the variable using the **Heuristics** like; minimum-remaining-values (MRV), degree heuristic (DH), or least-constraining-value (LCV).

Backtracking Search

```
function BACKTRACKING-SEARCH(csp) returns a solution or failure
  return BACKTRACK(csp, { })

function BACKTRACK(csp, assignment) returns a solution or failure
  if assignment is complete then return assignment
  var ← SELECT-UNASSIGNED-VARIABLE(csp, assignment)
  for each value in ORDER-DOMAIN-VALUES(csp, var, assignment) do
     if value is consistent with assignment then
        add {var = value} to assignment
        inferences ← INFERENCE(csp, var, assignment)
        if inferences ≠ failure then
           add inferences to csp
           result ← BACKTRACK(csp, assignment)
           if result ≠ failure then return result
           remove inferences from csp
        remove {var = value} from assignment
  return failure
```

Forward Checking:

The AC-3 can reduce the domains of variables before we begin the search. But inference can be even more powerful during the course of a search. One of the simplest forms of inference is called **forward checking**. The figure above shows the progress of backtracking search on the Australia CSP with forward checking.

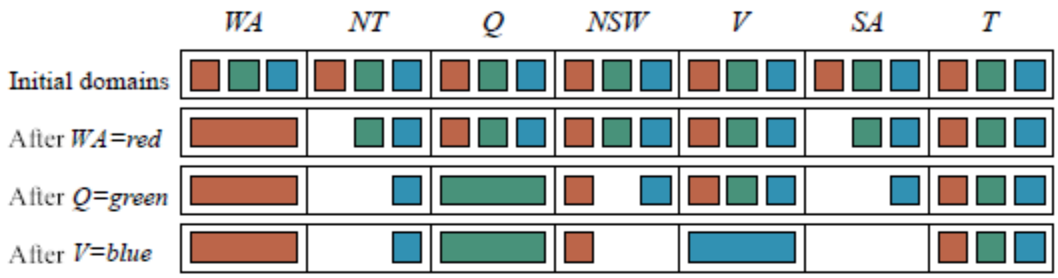

Figure The progress of a map-coloring search with forward checking. $WA = red$ is assigned first; then forward checking deletes red from the domains of the neighboring variables NT and SA. After $Q = green$ is assigned, $green$ is deleted from the domains of NT, SA, and NSW. After $V = blue$ is assigned, $blue$ is deleted from the domains of NSW and SA, leaving SA with no legal values.